🧪 TiDPI 00a — Zasady pracy na lekcji, odpowiedzialność i sprawozdania

📝 Tematy do dziennika

```
[TiDPI] Organizacja zajęć, zasady współpracy, sprawozdania oraz odpowiedzialne korzystanie z agentów AI
```

🎯 Cele lekcji

Po tej lekcji będziecie:

- wiedzieli, jak będą wyglądały zajęcia z testowania i dokumentowania projektów informatycznych,
- znali zasady pracy podczas części teoretycznej, ćwiczeń i projektów,
- rozumieli, dlaczego samo stwierdzenie „program działa” nie jest wystarczającym wynikiem testu,
- potrafili przygotować podstawowe sprawozdanie już od lekcji 01,
- znali zasady terminowego oddawania prac,
- wiedzieli, jak wolno korzystać z agentów AI i za co nadal odpowiadacie Wy jako autorzy pracy,
- rozumieli, jakie dane można przekazywać internetowym narzędziom AI.

👋 Zaczynamy nowy przedmiot

Zaczynamy od lekcji numerowanych `00a`, `00b` i `00c`. Dlaczego od zera? Dobry programista indeksuje od zera. A dlaczego litery? Ponieważ ten pierwszy blok ma trzy godziny i wyjątkowo wszystkie trzy godziny wypełni nam teoria więc będą do niej aż trzy dokumenty. Kolejne trzygodzinne bloki lekcyjne będą się odbywały według innego schamtu i do każdego bloku będzie jeden odrębny dokument.

Dzisiaj, podczas pierwszej godziny ustalimy zasady pracy. W drugiej odpowiemy na pytanie „dlaczego testujemy?”, a w trzeciej zbudujemy zgrubną mapę tego, co i jak można testować.

Znamy się już z zajęć z programowania, więc wiecie, że nie interesuje mnie bezmyślne przepisywanie kodu z tablicy. Na TiDPI pójdziemy o krok dalej. Nie będzie to kolejny kurs składni C++, Pythona ani obsługi środowiska programistycznego. Tym razem program stanie się obiektem małego informatycznego śledztwa.

Na tych zajęciach zajmiemy się pytaniami, które pojawiają się **po napisaniu programu**:

- Skąd wiemy, że program naprawdę działa poprawnie?
- Czy działa poprawnie również dla danych granicznych i błędnych?
- Jak zmierzyć jego czas działania lub zużycie pamięci?
- Jak wykonać pomiar tak, aby można go było powtórzyć?
- Jak udokumentować kod, test i wynik?
- Jak przekazać projekt innej osobie, aby mogła go uruchomić i sprawdzić?
- Jak odróżnić wynik pomiaru od naszej interpretacji tego wyniku?

Wyobraźcie sobie, że przekazujecie swój program koledze, który ma go uruchomić bez Waszej pomocy. Zdanie „u mnie działa” niewiele mu pomoże. Nie wie przecież, jakich danych użyliście, w jakiej wersji programu pracowaliście ani czego właściwie powinien się spodziewać. Dlatego w testowaniu i dokumentowaniu znacznie bardziej interesuje mnie inne zdanie:

> **Potrafię pokazać, co sprawdziłem, w jakich warunkach, jaki otrzymałem wynik i na jakiej podstawie wyciągnąłem wniosek.**

Jeżeli po zakończeniu kursu będziecie umieli uczciwie wypowiedzieć to zdanie o własnym projekcie, osiągniemy najważniejszy cel tych zajęć.

⏱️ Jak będą wyglądały zajęcia?

Jak już wiecie z planu zajęć, w tygodniu mamy trzy godziny lekcyjne. Chcę, aby tworzyły jeden spójny blok, w którym nowa wiedza od razu przechodzi w praktykę:

- **Pierwsza godzina — teoria** Wprowadzę nowe zagadnienie, wspólnie omówimy przykłady i przygotujemy metodę pracy.
- **Druga i trzecia godzina — praktyka** Oddaję stanowiska w Wasze ręce. Wykonujecie ćwiczenia, testy, pomiary, dokumentację albo pracujecie nad projektem. Ja w tym czasie pomagam, zadaję pytania i sprawdzam, czy obrana metoda rzeczywiście prowadzi do wiarygodnego wyniku.

Kalendarz czwartej klasy przypomina trasę z licznymi objazdami: praktyki zawodowe, mój urlop (bo ja nie mam wakacji, jak inni nauczyciele), święta, ferie i egzaminy będą regularnie przerywać naszą pracę. Dlatego jasno zaznaczę terminy i punkty kontrolne. Po dłuższej przerwie przypomnimy sobie najważniejsze rzeczy, ale sama przerwa w planie roku nie usuwa wcześniej ustalonych obowiązków. **Wyjątkowo w tym roku mimo przerw będziecie musieli dużo pracować poza lekcjami w domu.**

Dwa większe projekty

W ciągu roku zaproszę Was do dwóch większych projektów:

- **Projekt 1 — benchmark C++ i Pythona.** Po lekcji 07 rozpoczniecie w parach porównywanie podobnych rozwiązań napisanych w obu językach. Projekt zostanie zakończony i rozliczony w pierwszym półroczu, najpóźniej przy lekcji 12.
- **Projekt 2 — testowanie i dokumentowanie większego projektu.** W zespołach trzyosobowych wykorzystacie poznane techniki w dłuższej pracy projektowej. Szczegółowy temat, zakres i harmonogram otrzymacie przed rozpoczęciem projektu.

Podczas pracy projektowej nadal będę wykorzystywał pierwszą godzinę na wprowadzenie potrzebnego zagadnienia. Nie chcę jednak, aby teoria wisiała w próżni — na kolejnych godzinach zastosujecie ją w ćwiczeniu albo we własnym projekcie.

🤝 Zasady współpracy na lekcji

Przychodzicie po zapoznaniu się z materiałem

Przed każdą lekcją zapoznajcie się z przeznaczonym do niej materiałem. Nie musicie od razu wszystkiego rozumieć ani znać odpowiedzi na każde pytanie — właśnie po to spotykamy się na zajęciach. Zaznaczcie jednak fragmenty niejasne, zanotujcie wątpliwości i przygotujcie pytania do prowadzącego. Dzięki temu wykład nie będzie pierwszym spotkaniem z tematem, lecz okazją do uporządkowania wiedzy, wyjaśnienia trudności i wspólnej dyskusji.

> Przeczytanie materiału przed lekcją nie sprawi, że automatycznie wszystko stanie się proste. Sprawi za to, że będziecie wiedzieli, w którym miejscu przestało być proste — a to już bardzo cenna informacja diagnostyczna.

Realizujemy program systematycznie

Kolejne lekcje tworzą jeden ciąg i wykorzystują umiejętności zdobyte wcześniej. Jeżeli zajęcia zostaną odwołane z powodu mojej choroby, urlopu albo innej nieobecności, realizujecie zaplanowany materiał samodzielnie, we własnym zakresie i na miarę swoich możliwości w domu. Zapoznajcie się z teorią, uruchomcie dostępne

przykłady, wykonajcie tyle ćwiczeń, ile możecie, oraz zapiszcie pytania i napotkane problemy. Dzięki temu po powrocie nie będziemy tracili kolejnego tygodnia na rozpoczynanie całego bloku od zera.

Samodzielna realizacja odwołanego bloku nie oznacza, że macie bez pomocy rozwiązać każdą trudność. Oznacza, że podejmujecie pracę, zachowujecie jej stan i potraficie pokazać, gdzie pojawiła się przeszkoda. Do pytań i nierozwiązanych problemów wrócimy na najbliższych zajęciach.

Kiedy mówię ja albo ktoś z klasy, pozostali słuchają

Gdy wyjaśniam zagadnienia, pokazuję przykład, przekazuję polecenie albo odpowiadam na czyjeś pytanie, Wy słuchacie i nie prowadzicie rozmów między sobą. Ta sama zasada obowiązuje podczas wypowiedzi ucznia skierowanej do całej klasy.

Nie oczekuję bezwzględnej ciszy przez całe zajęcia. To byłoby zresztą dość dziwne na przedmiocie, na którym będziecie porównywać pomysły i wspólnie szukać błędów. W części praktycznej będziecie:

- zadawać pytania,
- porównywać metody,
- pomagać sobie w odnalezieniu błędu,
- omawiać wyniki w parze lub zespole.

Rozmawiajcie więc wtedy, kiedy rozmowa pomaga Wam pracować. Jeżeli zagłusza moje wyjaśnienie albo przeszkadza innej osobie w skupieniu, przestaje być pomocą.

Pytania są częścią pracy

Jeżeli nie rozumiecie polecenia, pojęcia, wyniku albo zachowania programu — pytajcie mnie. Dobre pytanie zadane odpowiednio wcześnie oszczędza czas całej klasie. Wolę zatrzymać się na dwie minuty i coś wyjaśnić, niż po godzinie odkryć, że połowa klasy testowała nie ten program albo mierzyła nie tę wartość.

Możecie również zakwestionować mój przykład lub wniosek. Jeżeli znajdziecie przypadek, który mu przeczy, pokażcie go. W testowaniu nie bronimy przypuszczenia za wszelką cenę — szukamy danych, które pozwolą je potwierdzić albo odrzucić. Czasami najlepszym wynikiem lekcji będzie właśnie dobrze udowodnione: „Proszę pana, ale tutaj to nie działa”.

Pomoc nie oznacza kopiowania

Będę Was zachęcał do rozsądnej pomocy. Możecie wyjaśnić koledze pojęcie, wskazać błąd lub zaproponować przypadek testowy. Nie możecie jednak przedstawiać cudzego kodu, pomiaru, opisu ani wniosku jako własnego.

W ćwiczeniach indywidualnych każdy z Was:

- sam wykonuje wymagane testy,
- zachowuje własne wyniki,
- przygotowuje własne sprawozdanie,
- potrafi wyjaśnić każdy oddany element.

W projektach zespołowych odpowiedzialność jest wspólna, ale poproszę Was o czytelny podział pracy. Każdy członek zespołu powinien rozumieć cały oddawany rezultat, a szczególnie własną część. Zdanie „to robił kolega, więc nie wiem” nie będzie dobrą odpowiedzią podczas omawiania projektu.

🔎 „Działa” to dopiero początek

Klikacie „Uruchom”, program nie wyświetla błędu, a na ekranie pojawia się liczba. Kuszące jest wtedy postawienie wielkiego zielonego znaku: „gotowe”. Niestety, uruchomienie programu bez komunikatu o błędzie nie dowodzi jeszcze jego poprawności.

Spójrzmy na prosty program obliczający średnią ocen. Dla danych `3, 4, 5, 4, 4` może podać poprawny wynik, a jednocześnie:

- dzielić przez zero dla pustego zestawu,
- przyjmować oceny spoza dozwolonego zakresu,
- tracić część ułamkową przez nieprawidłowy typ danych,
- podawać poprawny wynik tylko przypadkiem,
- działać zbyt wolno dla dużego zestawu danych.

Dlatego w sprawozdaniu rozdzielamy trzy rzeczy:

- **Dane i warunki** — czego użyliście i w jakim środowisku wykonaliście test.
- **Wynik** — co rzeczywiście zwrócił program lub pokazał pomiar.
- **Wniosek** — jak interpretujecie wynik i co z niego wynika.

> Wynik: „czas wykonania wyniósł 38 ms” jest zapisem obserwacji. Wniosek: „dla badanego zestawu wariant A był szybszy od wariantu B” jest interpretacją porównanych wyników.

Nie zmieniajcie ani nie wybierajcie wyników tylko dlatego, że nie pasują do oczekiwanej odpowiedzi. Jeżeli spodziewaliście się 10 ms, a otrzymaliście 80 ms, nie chowajcie wyniku pod dywan. Taka różnica może wskazywać błąd programu, metody pomiaru albo założenia — i właśnie dlatego jest wartościowa.

📄 Sprawozdania z lekcji 01–07

Po każdej lekcji praktycznej od 01 do 07 przygotujecie **jedno indywidualne sprawozdanie**. Każde z nich sprawdzę osobno. Dzięki temu zobaczę nie tylko końcowy wynik, lecz także to, jak planujecie pomiar, zapisujecie dane i poprawiacie własny sposób pracy.

Na każdej z tych lekcji otrzymacie ode mnie:

- **zakres podstawowy**, wspólny dla całej klasy,
- **osobną część rozszerzającą** dla osób, które zakończą podstawę wcześniej albo chcą zbadać problem dokładniej.

Rozszerzenia opisujecie w tym samym sprawozdaniu i wyraźnie wskazujecie, które elementy wykonaliście. Nie musicie udawać, że zrobiliście więcej, niż zdążyliście. Zdecydowanie wyżej ocenię uczciwie wykonany zakres podstawowy niż efektowną tabelę z wynikami pomiaru, który nigdy się nie odbył.

Lekcja 01 przygotuje Was do tworzenia sprawozdań

Dokumentowanie zaczniemy od razu. Podczas lekcji 01 poznacie Markdown, rozbudujecie prosty szablon zamieszczony niżej i przygotujecie kompletne sprawozdanie na podstawie fikcyjnego pakietu wyników. Nauczycie się porządkować dane w tabelach, podpisywać dowody, odróżniać wynik od interpretacji oraz generować z jednego pliku źródłowego dokumenty DOCX i PDF za pomocą Pandoca.

Dzięki temu sprawozdania z ćwiczeń wykonywanych podczas lekcji 02–07 nie będą pierwszą próbą jednoczesnego przeprowadzania eksperymentu i wymyślania sposobu jego opisania. Będziecie korzystali z poznanego układu raportu, dostosowując jego zawartość do konkretnego zadania, pomiaru i wymaganych dowodów.

W każdym sprawozdaniu najważniejsze pozostają:

- kompletność podstawowych informacji,
- zapis rzeczywiście wykonanych kroków,
- czytelne przedstawienie wyników,
- odróżnienie wyniku od wniosku,
- zgodność opisu z dostarczonymi plikami i z tym, co potraficie wyjaśnić.

⏰ Termin nadal jest częścią zadania

Powiem o tym wprost, ponieważ ta zasada będzie obowiązywała przez cały rok. W pracy zawodowej wynik dostarczony po terminie może być bezużyteczny, nawet jeżeli technicznie jest poprawny. Raport z testu przesłany po wydaniu programu przypomina informację o dziurze w moście przekazaną po przejeździe ciężarówki. Na TiDPI uczymy się tej samej odpowiedzialności, ale pojedyncze potknięcie nie będzie jeszcze automatycznie oznaczało oceny niedostatecznej.

> **Ta zasada dotyczy siedmiu indywidualnych sprawozdań po lekcjach 01–07. Sprawozdanie niedostarczone w wyznaczonym terminie zapisuję jako brak oceny. Każde dwa brakujące sprawozdania — niezależnie od tego, czy nastąpiły po sobie — skutkują jedną oceną niedostateczną.**

Przelicznik jest prosty:

- jedno brakujące sprawozdanie — jeden brak oceny, ale jeszcze bez oceny niedostatecznej,
- dwa brakujące sprawozdania — jedna ocena niedostateczna,
- cztery brakujące sprawozdania — dwie oceny niedostateczne,
- sześć brakujących sprawozdań — trzy oceny niedostateczne.

Braki sumują się wyłącznie w cyklu siedmiu indywidualnych sprawozdań po lekcjach 01–07 i nie muszą występować na dwóch kolejnych lekcjach. Trzeci brak rozpoczyna więc następną parę, czwarty ją zamyka i powoduje wpisanie kolejnej oceny niedostatecznej.

Sprawozdania nadal oddajecie w wyznaczonym terminie. Nie ma zwykłego trybu dostarczenia tej samej pracy później i usunięcia w ten sposób zapisanego braku.

Po lekcji 07 kończy się cykl indywidualnych sprawozdań. Późniejsze projekty będziecie wykonywać w zespołach dwu- i trzyosobowych. Nie stosuję do nich opisanego wyżej przelicznika braków. Każdy projekt otrzyma własne zasady oceniania, terminy i punkty kontrolne, które omówię z Wami przed rozpoczęciem pracy.

Dla każdego zadania podam Wam termin oraz sposób uznania pracy za dostarczoną. Od tej chwili to Wy — indywidualnie albo jako zespół — odpowiadacie za:

- zaplanowanie pracy,
- pozostawienie czasu na sprawdzenie plików,
- wykonanie kopii zapasowej,
- dostarczenie kompletnej wersji przed upływem terminu,
- upewnienie się, że pliki można otworzyć i odczytać.

Uszkodzony nośnik, brakujący załącznik albo plik, którego nie da się otworzyć, nie jest kompletnym oddaniem pracy. Szczególne sytuacje będę rozpatrywał zgodnie z przekazanymi Wam zasadami i obowiązującymi przepisami szkolnymi — nie zakładajcie samodzielnie, że termin został przesunięty. Jeżeli wydarzy się coś nadzwyczajnego, poinformujcie mnie możliwie szybko.

🤖 Agenci AI są dozwoleni — odpowiedzialność pozostaje po Waszej stronie

Na zajęciach możecie korzystać z agentów AI, między innymi z Codexa lub GitHub Copilota. Co więcej, będę Was zachęcał, abyście uczyli się z nimi pracować. Umiejętne zadanie pytania, sprawdzenie odpowiedzi i poprawienie błędu agenta staje się jedną z kompetencji współczesnego programisty.

Agent AI może pomóc Wam:

- wyjaśnić niezrozumiałe pojęcie,
- zaproponować przypadki testowe,
- znaleźć możliwą przyczynę błędu,
- przejrzeć kod lub strukturę dokumentu,
- zaproponować czytelniejszy opis,
- wskazać pytania, które warto zadać otrzymanym wynikom.

Agent AI może również:

- podać nieprawdziwą informację,
- wymyślić wynik pomiaru, którego nie wykonaliście,
- nie zauważyć ważnego przypadku brzegowego,
- zaproponować kod, którego nie rozumiecie,
- wyciągnąć zbyt daleko idący wniosek,
- ujawnić w odpowiedzi dane przesłane przez użytkownika lub wykorzystać je w sposób, którego nie kontrolujecie.

Dlatego obowiązuje prosta zasada:

> **Możecie korzystać z AI, ale podpisujecie się tylko pod materiałem, który rozumiecie, sprawdziliście i potraficie obronić.**

Mogę zapytać Was o dowolny fragment kodu, tabelę, pomiar, termin lub wniosek znajdujący się w oddanej pracy. Nie będę obniżał oceny za samo uczciwe użycie AI. Jeżeli jednak nie potraficie wyjaśnić własnego sprawozdania lub projektu, oznacza to, że nie wykazaliście samodzielnego wykonania i zrozumienia pracy. Ocena może wtedy zostać drastycznie obniżona, aż do oceny niedostatecznej.

Nie jest problemem samo użycie AI. Problemem jest:

- bezrefleksyjne skopiowanie odpowiedzi,
- przedstawienie wymyślonych danych jako pomiarów,
- zatajenie błędu zauważonego podczas weryfikacji,
- oddanie materiału, którego autor nie rozumie.

Jak weryfikować pomoc AI?

Potraktujcie odpowiedź agenta jak podpowiedź kolegi, który mówi bardzo pewnym głosem, ale czasami zmyśla. Zanim ją wykorzystacie:

- przeczytajcie ją w całości,
- sprawdźcie, czy odpowiada dokładnie na Wasze pytanie,
- uruchomcie proponowany kod lub test w kontrolowanych warunkach,
- porównajcie twierdzenia z dokumentacją, kodem i własnymi wynikami,
- poprawcie zauważone błędy,
- upewnijcie się, że potraficie wyjaśnić rezultat własnymi słowami.

W sprawozdaniu krótko wskażcie, z jakiego narzędzia lub źródła korzystaliście, do czego Wam pomogło i jak zweryfikowaliście uzyskaną pomoc. Nie wklejajcie całej rozmowy z agentem, jeżeli polecenie nie wymaga jej jako osobnego załącznika.

🔐 Ochrona danych i kryptonim ucznia

Wasze sprawozdania mogę analizować przy wsparciu internetowego agenta AI. Dlatego w materiałach przekazywanych takiemu narzędziu **nie umieszczamy imienia, nazwiska ani innych niepotrzebnych danych osobowych**.

Każdemu z Was przydzielę indywidualny kryptonim TiDPI, który powiążę z właściwą osobą w prywatnym zestawieniu nauczyciela. Dzięki temu agent zobaczy wyłącznie kryptonim, a ja nadal będę wiedział, czyją pracę oceniam. Nie wymyślajcie własnych kryptonimów i nie zamieniajcie ich między sobą. Obowiązujący wzorzec nazw plików podam Wam osobno.

W materiałach przeznaczonych do analizy przez AI:

- używajcie wyłącznie przydzielonego kryptonimu TiDPI,
- nie umieszczajcie imienia ani nazwiska w treści, nazwach plików, zrzutach ekranu i metadanych,
- nie przekazujcie haseł, kluczy dostępu, prywatnych repozytoriów ani cudzych danych,
- sprawdzajcie zrzuty ekranu — nazwa konta systemowego może ujawnić dane osobowe,
- nie umieszczajcie w publicznym repozytorium materiałów, których nie wolno publikować.

Anonimizacja nie oznacza anonimowości wobec mnie. Nadal będę mógł przypisać pracę do właściwej osoby i wpisać ocenę do dziennika. Chronimy dane przed niepotrzebnym przekazaniem ich narzędziu internetowemu, a nie ukrywamy autora pracy przed nauczycielem.

🛡️ Testujemy tylko to, do czego mamy prawo

Nie każdy technicznie możliwy test jest dozwolony. To, że potraficie sprawdzić cudze zabezpieczenie albo zasypać serwis tysiącem żądań, nie oznacza jeszcze, że wolno Wam to zrobić.

Na zajęciach testujecie:

- własne programy,
- programy i dane udostępnione do ćwiczenia,
- projekty, dla których otrzymaliście wyraźną zgodę,
- środowiska wskazane przeze mnie.

Nie testujecie bez pozwolenia cudzych kont, komputerów, sieci, stron internetowych, usług ani zabezpieczeń. Nie wysyłacie masowych żądań i nie próbujecie „sprawdzić”, czy obcy system da się uszkodzić lub ominąć.

Jeżeli nie jesteście pewni, czy planowany test jest bezpieczny i dozwolony — najpierw pytacie mnie. W tym przypadku lepiej zadać jedno pytanie za dużo niż wykonać jeden test za daleko.

🧰 Narzędzia pracy

Na zajęciach będziemy korzystać przede wszystkim z:

- **Visual Studio** — głównie do projektów w C i C++,
- **Visual Studio Code** — do kodu, plików Markdown, skryptów i dokumentacji,
- **PyCharm** — opcjonalnie do projektów w Pythonie,
- **Git** — do zapisywania historii zmian,
- **GitHub** — do przechowywania i współdzielenia wybranych projektów oraz późniejszej automatyzacji testów,
- narzędzi systemowych i prostych skryptów uruchamianych lokalnie.

Nie będę od Was wymagał stawiania własnych serwerów ani wykonywania rozbudowanych testów aplikacji webowych lub mobilnych. Skupimy się na programach i narzędziach, które możecie bezpiecznie uruchomić na szkolnych komputerach.

Wasza praca potrzebuje co najmniej dwóch bezpiecznych miejsc

Nie chcę usłyszeć w dniu oddania: „Miałem wszystko na pendrivie, ale pendrive przestał działać". Urządzenia się psują, pliki bywają przypadkowo usuwane, a szkolny komputer nie jest prywatnym archiwum. Nie przechowujcie więc jedynego egzemplarza pracy na pulpicie szkolnego komputera ani na jednym nośniku. Korzystajcie z co najmniej dwóch miejsc, na przykład:

- folderu roboczego na komputerze,
- własnego nośnika,
- właściwie zabezpieczonego repozytorium Git,
- innego miejsca zaakceptowanego przeze mnie.

Sposób przekazania konkretnego sprawozdania oraz obowiązujący wzorzec nazwy pliku podam w poleceniu. Wszystkie elementy pracy umieszczajcie w jednym, uporządkowanym folderze, o ile nie określę inaczej.

📜 Minimalny szablon sprawozdania od lekcji 01

Daję Wam poniższy szablon już teraz, aby od pierwszego ćwiczenia było wiadomo, co i gdzie zapisać. Skopiujcie go do pliku Markdown z rozszerzeniem `.md`, a następnie zastąpcie podpowiedzi własną treścią.

Nie zostawiajcie w gotowym sprawozdaniu zdań typu „wpisz wynik tutaj". Jeżeli dana sekcja nie dotyczy ćwiczenia, napiszcie krótko dlaczego, zamiast usuwać ją bez wyjaśnienia.

```
# Sprawozdanie TiDPI – ćwiczenie XX

## Dane pracy

- Kryptonim ucznia: `KRYPTONIM_TIDPI`
- Numer i temat lekcji: `XX – temat`
- Wykonany zakres: `podstawowy` / `podstawowy + rozszerzenia ...`
- Data wykonania testów: `RRRR-MM-DD`

## 1. Cel

Napiszcie własnymi słowami, co sprawdzaliście lub mierzyliście.

## 2. Środowisko testowe

| Element | Wartość |
|---|---|
| System operacyjny | ... |
| Język i wersja kompilatora/interpretera | ... |
| Tryb kompilacji, jeżeli dotyczy | ... |
| Procesor lub inne istotne parametry | ... |
| Wykorzystane narzędzia | ... |
```

````
## 3. Badany program i dane

Opiszcie krótko program, użyte dane wejściowe oraz sposób ich przygotowania.

Jeżeli potrzebny jest krótki fragment kodu, wstawcie go jako tekst:

```cpp
// Tylko fragment istotny dla opisywanego testu.
```

Pełny kod umieśćcie w załączonych plikach źródłowych.

## 4. Metoda i przebieg

1. Opiszcie pierwszy wykonany krok.
2. Opiszcie kolejne kroki w takiej kolejności, aby test dało się powtórzyć.
3. Podajcie liczbę powtórzeń pomiaru, jeżeli wykonywaliście pomiar wielokrotnie.

## 5. Wyniki

Wstawcie rzeczywiście uzyskane wyniki. Dobierzcie kolumny tabeli do ćwiczenia.

| Przypadek lub próba | Dane wejściowe | Wynik oczekiwany | Wynik otrzymany | Uwagi |
|---|---:|---:|---:|---|
| 1 | ... | ... | ... | ... |
| 2 | ... | ... | ... | ... |

Nie wpisujcie wyników, których nie uzyskaliście podczas własnego testu.

## 6. Sprawdzenie poprawności

Wyjaśnijcie, jak sprawdziliście, czy otrzymane wyniki są wiarygodne i zgodne z oczekiwaniami.

## 7. Analiza

Opiszcie zauważone zależności, różnice, nietypowe wyniki i możliwe źródła błędów.

## 8. Wnioski

Zapiszcie krótko, co wynika z testu. Nie powtarzajcie samej tabeli wyników.

## 9. Wykorzystanie AI i innych źródeł

- Narzędzie lub źródło: ...
- W czym pomogło: ...
- Jak sprawdziliście otrzymaną pomoc: ...

Jeżeli nie korzystaliście z AI ani dodatkowych źródeł, napiszcie to wprost.

## 10. Załączniki

- `src/...` – pliki źródłowe,
- `dane/...` – dane wejściowe lub wyniki,
- `obrazy/...` – potrzebne zrzuty ekranu,
- inne pliki wymagane w poleceniu.
````

Co jest dowodem, a co tylko ozdobą?

Zależy mi na dowodach, które można odczytać, sprawdzić i — jeśli będzie taka potrzeba — wykorzystać ponownie. Dobrym dowodem może być:

- kod źródłowy możliwy do uruchomienia,
- plik z danymi wejściowymi,
- zapis wyników programu,
- tabela surowych pomiarów,
- log z kompilacji lub testu,
- zrzut ekranu pokazujący szczegół trudny do zapisania jako tekst.

Sam zrzut ekranu kodu nie zastępuje pliku źródłowego. Zdjęcie tabeli nie jest lepsze od tabeli zapisanej jako tekst. Do sprawozdania dołączajcie tylko te obrazy, które faktycznie pomagają udowodnić wynik lub wyjaśnić problem.

✅ Kontrola przed oddaniem pracy

Przed przekazaniem sprawozdania sprawdźcie:

☐ Czy użyliście przydzielonego kryptonimu TiDPI zamiast imienia i nazwiska?
☐ Czy nazwa pliku lub folderu jest zgodna z bieżącym wzorcem, który Wam podałem?
☐ Czy wykonaliście cały wymagany zakres podstawowy?
☐ Czy zaznaczyliście wykonane rozszerzenia?
☐ Czy opisaliście cel, środowisko, metodę, wyniki, analizę i wnioski?
☐ Czy wyniki pochodzą z Waszych rzeczywistych testów?
☐ Czy pełny kod znajduje się w plikach źródłowych, a nie wyłącznie na zrzutach ekranu?
☐ Czy wszystkie odwołania do załączników działają po przeniesieniu całego folderu?
☐ Czy usunęliście zbędne dane osobowe, hasła i klucze dostępu?
☐ Czy wskazaliście wykorzystanie AI i innych źródeł oraz sposób weryfikacji?
☐ Czy potraficie wyjaśnić każdy fragment oddanej pracy?
☐ Czy otworzyliście końcową kopię i sprawdziliście ją przed terminem?

📚 Słowniczek

Testowanie	zaplanowane sprawdzanie programu w celu wykrycia błędów i zebrania informacji o jego jakości.
Przypadek testowy	opis danych wejściowych, warunków wykonania i oczekiwanego wyniku pojedynczego testu.
Pomiar	wyznaczenie wartości określonej cechy, na przykład czasu działania lub zużycia pamięci, według ustalonej metody.
Dowód	materiał pozwalający potwierdzić, że określony test lub pomiar został wykonany i dał zapisany wynik.
Sprawozdanie	uporządkowany dokument opisujący cel, warunki, przebieg, wyniki, analizę i wnioski z wykonanej pracy.
Benchmark	kontrolowany test służący do pomiaru i porównania wydajności programów, algorytmów albo konfiguracji.
Deadline	ostateczny termin dostarczenia pracy. Po jego upływie praca jest spóźniona.
Agent AI	narzędzie wykorzystujące sztuczną inteligencję, które może wykonywać wieloetapowe zadania, na przykład analizować kod, proponować testy lub pomagać redagować dokumentację.
Weryfikacja	sprawdzenie, czy informacja, kod, wynik lub wniosek jest poprawny i zgodny z wymaganiami.
Anonimizacja	usunięcie albo zastąpienie danych pozwalających bez potrzeby rozpoznać konkretną osobę.
Repozytorium Git	folder projektu wraz z zapisaną historią zmian jego plików.

🎓 Podsumowanie

Na TiDPI nie wystarczy uruchomić programu i powiedzieć mi, że działa. Będę od Was wymagał zaplanowania testu, zebrania dowodów, przeanalizowania wyników i przygotowania dokumentacji, którą inna osoba może zrozumieć oraz zweryfikować.

Najważniejsze zasady na cały rok:

- **Przed każdą lekcją zapoznajecie się z materiałem i przygotowujecie pytania do prowadzącego.**
- **Realizujecie program systematycznie; jeśli zajęcia zostaną odwołane z powodu mojej choroby, urlopu albo innej nieobecności, pracujecie nad zaplanowanym materiałem samodzielnie, we własnym zakresie i na miarę swoich możliwości.**
- **Słuchacie, gdy mówię do Was ja albo gdy inna osoba zwraca się do całej klasy.**
- **Każde ćwiczenie od 01 do 07 kończy się indywidualnym sprawozdaniem.**
- **Wynik musi pochodzić z rzeczywiście wykonanej pracy.**
- **W cyklu lekcji 01–07 każde dwa brakujące sprawozdania oznaczają jedną ocenę niedostateczną. Projekty mają własne zasady oceniania.**
- **Możecie korzystać z AI, ale musicie rozumieć i zweryfikować oddawany materiał.**
- **Chronimy dane osobowe i testujemy wyłącznie systemy, do których mamy uprawnienia.**
- **Pytamy, gdy czegoś nie rozumiemy, i uczciwie opisujemy napotkane problemy.**

> Dobre testowanie nie polega na udowadnianiu, że nie popełniliśmy błędu. Polega na takim sprawdzaniu programu, aby błąd miał możliwie małą szansę pozostać niezauważony.